Министерство образования Республики Беларусь
Учреждение образования
«Брестский государственный технический университет»
Факультет электронно-информационных систем
Кафедра интеллектуальных информационных технологий

Отчет по лабораторной работе №2

Специальность АС-68

Выполнил:
Студент 3-го курса
Группы АС-68
Г. В. Пошвенчук
Проверил:
А. А. Крощенко
Ст. преп. кафедры ИИТ,
«___»_____ 2026 г.

Брест 2026

Цель работы: изучить применение линейной и логистической регрессии для решения практических задач. Научиться обучать модели, оценивать их качество с помощью соответствующих метрик и интерпретировать результаты.

Вариант 6

1. Регрессия (Прогнозирование цен на жильё в Бостоне)

- Датасет: Boston Housing
- Целевая переменная: Предсказать медианную стоимость дома (MEDV)
- Задания:
  - загрузите данные. Используйте все признаки для обучения;
  - обучите модель линейной регрессии;
  - рассчитайте MSE и R2;
  - визуализируйте зависимость MEDV от RM (среднее количество комнат) с линией регрессии.

2. Классификация (Диагностика рака груди)

- Датасет: Breast Cancer Wisconsin
- Целевая переменная: Предсказать, является ли опухоль злокачественной (malignant) или доброкачественной (benign)
- Задания:
  - загрузите данные и выполните стандартизацию признаков;
  - обучите модель логистической регрессии;
  - рассчитайте Accuracy, Precision и Recall;
  - постройте матрицу ошибок и объясните значение каждого из её квадрантов в контексте задачи.

Код программы 1:

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

# 1. Загрузка
df = pd.read_csv("BostonHousing.csv")

# Убираем служебные и производные столбцы (CAT. MEDV — утечка целевой
переменной)
df = df.drop(columns=[c for c in ["Unnamed: 0", "CAT. MEDV"] if c in
df.columns])
df = df.dropna()

# 2. Разделение 80/20
test = df.sample(frac=0.2, random_state=42)
train = df.drop(test.index)

# Формирование матриц признаков (X) и целевых переменных (y) для train и
test
features = [c for c in df.columns if c != "MEDV"]
X_train = train[features].to_numpy(dtype=float)
X_test = test[features].to_numpy(dtype=float)
y_train = train["MEDV"].to_numpy(dtype=float)
y_test = test["MEDV"].to_numpy(dtype=float)

# 3. Линейная регрессия (столбец единиц для свободного члена)
add_bias = lambda X: np.column_stack([np.ones(len(X)), X])
beta, *_ = np.linalg.lstsq(add_bias(X_train), y_train, rcond=None)
y_pred = add_bias(X_test) @ beta

# 4. Метрики
```

```
      # Расчет метрик качества (ошибка MSE и коэффициент детерминации R2)
      mse = np.mean((y_test - y_pred) ** 2)
      r2 = 1 - np.sum((y_test - y_pred) ** 2) / np.sum((y_test -
y_test.mean()) ** 2)
      print(f"MSE: {mse:.3f}")
      print(f"R2 : {r2:.3f}")

      # Создание таблицы для наглядного сравнения реальных и предсказанных цен
      results = pd.DataFrame({
          'Реальная цена (тыс. $)': y_test,
          'Предсказанная цена': y_pred.round(2),
          'Разница': (y_test - y_pred).round(2)
      })
      print(results.sample(n=10, random_state=42))

      # 5. Scatter MEDV от RM + линия регрессии
      # Построение линии тренда для графика (отдельная модель на 1 признаке)
      k, b = np.polyfit(df["RM"], df["MEDV"], 1)
      x_line = np.linspace(df["RM"].min(), df["RM"].max(), 100)

      # Визуализация данных и линии регрессии
      plt.figure(figsize=(8, 6))
      plt.scatter(df["RM"], df["MEDV"], alpha=0.6, color="steelblue")
      plt.plot(x_line, k * x_line + b, color="black", linewidth=2,
label="Линия регрессии")
      plt.xlabel("RM (среднее число комнат)")
      plt.ylabel("MEDV (медианная стоимость, $1000)")
      plt.title("Зависимость MEDV от RM")
      plt.legend()
      plt.grid(alpha=0.3)
      plt.show()
```

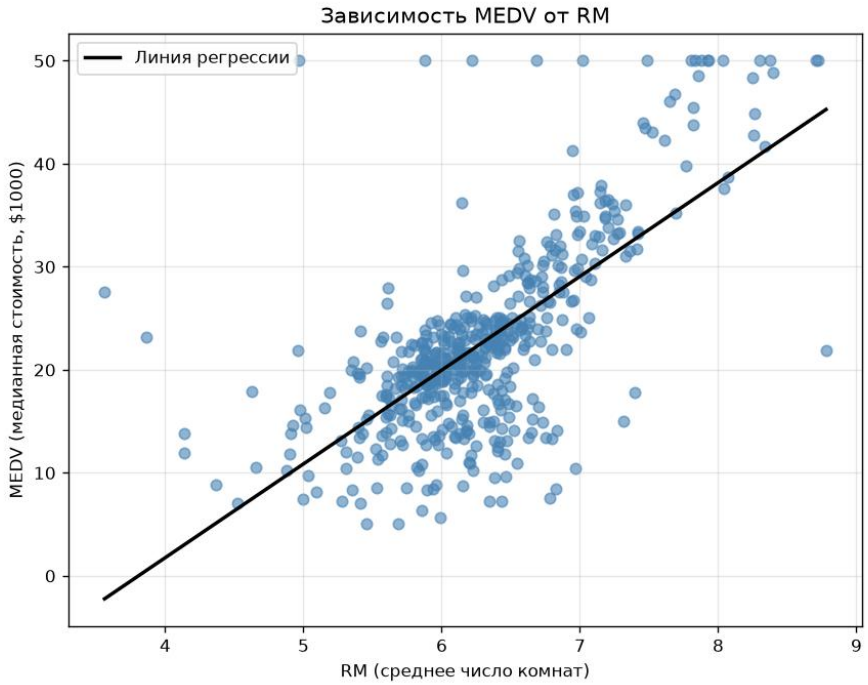
Результат:

MSE: 22.881

R2 : 0.691

	Реальная цена (тыс. $)	Предсказанная цена	Разница
84	19.4	23.12	-3.72
55	21.2	21.54	-0.34
66	13.1	19.24	-6.14
67	24.8	25.96	-1.16
45	15.1	15.85	-0.75
39	23.4	24.65	-1.25
22	12.7	11.49	1.21
44	22.6	27.36	-4.76
10	21.5	24.79	-3.29
0	23.6	28.96	-5.36

Визуализация:



Код программы 2:

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

# 1. Загрузка
df = pd.read_csv("breast_cancer.csv")

# Убираем служебные/пустые столбцы
df = df.drop(columns=[c for c in df.columns if c.startswith("Unnamed")
or c == "id"])

# diagnosis -> target
if "diagnosis" in df.columns:
    df = df.rename(columns={"diagnosis": "target"})

# M/B -> 0/1 (malignant = 0, benign = 1)
df["target"] = df["target"].astype(str).str.strip().map({"M": 0, "B":
1})
assert df["target"].notna().all(), "В target остались NaN — проверьте
CSV"
df["target"] = df["target"].astype(int)
df = df.dropna()

# 2. Разделение 80/20
rng = np.random.default_rng(42)
idx = rng.permutation(len(df))
n_test = int(0.2 * len(df))
test_idx, train_idx = idx[:n_test], idx[n_test:]

# Формирование матриц признаков (X) и целевых переменных (y) для train и
test
X = df.drop(columns=["target"])
y = df["target"].to_numpy()
X_train, X_test = X.iloc[train_idx].copy(), X.iloc[test_idx].copy()
y_train, y_test = y[train_idx], y[test_idx]

# 3. Стандартизация по train
# Вычисление среднего и стандартного отклонения только по обучающей
выборке
mean, std = X_train.mean(), X_train.std()
X_train = (X_train - mean) / std
X_test = (X_test - mean) / std
```

```
    # Добавление столбца единиц для свободного члена
    add_bias = lambda A: np.column_stack([np.ones(len(A)),
A.to_numpy(dtype=float)])
    Xb_train, Xb_test = add_bias(X_train), add_bias(X_test)

    # 4. Логистическая регрессия (градиентный спуск, L2-регуляризация)
    # Обучение модели: 5000 итераций градиентного спуска для поиска весов
    sigmoid = lambda z: 1 / (1 + np.exp(-z))
    w = np.zeros(Xb_train.shape[1])
    lr, lam = 0.1, 0.01
    for _ in range(5000):
        p = sigmoid(Xb_train @ w)
        grad = Xb_train.T @ (p - y_train) / len(y_train)
        reg = lam * w
        reg[0] = 0 # свободный член не регуляризуем
        w -= lr * (grad + reg)

    y_pred = (sigmoid(Xb_test @ w) >= 0.5).astype(int)

    # 5. Матрица ошибок и метрики (положительный класс — malignant = 0)
    y_true_pos = (y_test == 0).astype(int)
    y_pred_pos = (y_pred == 0).astype(int)

    # Расчет метрик качества классификации (TP, TN, FP, FN)
    tp = np.sum((y_pred_pos == 1) & (y_true_pos == 1))
    tn = np.sum((y_pred_pos == 0) & (y_true_pos == 0))
    fp = np.sum((y_pred_pos == 1) & (y_true_pos == 0))
    fn = np.sum((y_pred_pos == 0) & (y_true_pos == 1))

    # Вычисление Accuracy, Precision, Recall и F1-score
    accuracy = (tp + tn) / len(y_test)
    precision = tp / (tp + fp)
    recall = tp / (tp + fn)
    f1 = 2 * precision * recall / (precision + recall)

    print(f"Общая точность: {accuracy:.4f}")
    print(f"Точность предсказания рака: {precision:.4f}")
    print(f"Полнота (сколько найдено больных раком): {recall:.4f}")

    # Визуализация матрицы ошибок
    cm = np.array([[tn, fp], [fn, tp]])
    fig, ax = plt.subplots(figsize=(6, 5))
    ax.imshow(cm, cmap="Blues")
    ax.set_xticks([0, 1], ["Доброкачественная", "Злокачественная"])
    ax.set_yticks([0, 1], ["Доброкачественная", "Злокачественная"])
    ax.set_xlabel("Предсказано моделью")
    ax.set_ylabel("Истинный диагноз")
    ax.set_title("Матрица ошибок — Рак груди")

    # Вывод цифр в ячейках
    for i in range(2):
        for j in range(2):
            ax.text(j, i, cm[i, j], ha="center", va="center", fontsize=16,
                    color="white" if cm[i, j] > cm.max() / 2 else "black")

    plt.tight_layout()
    plt.show()
```

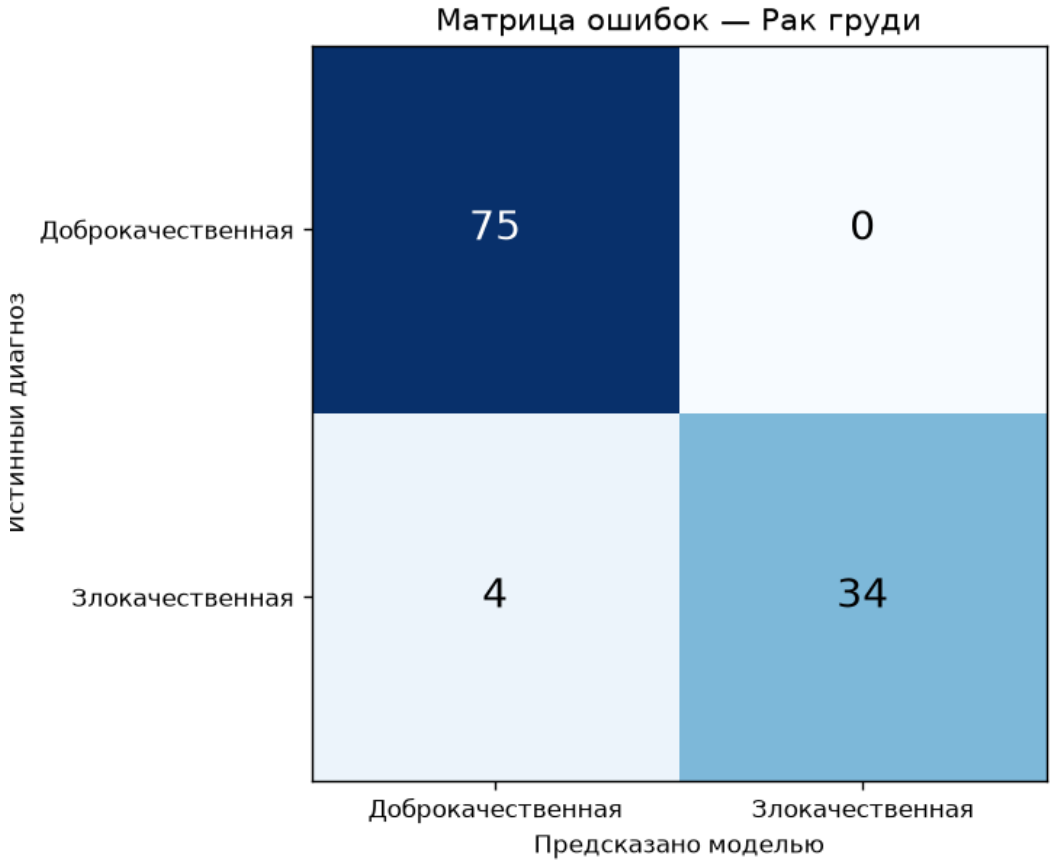
Результат:

Общая точность: 0.9646

Точность предсказания рака: 1.0000

Полнота (сколько найдено больных раком): 0.8947

Визуализация:



Вывод: изучено применение линейной и логистической регрессии для решения практических задач. Получены навыки по обучению моделей, оцениванию их качества с помощью соответствующих метрик и интерпретации результатов.